

Module – 1

Introduction to Database Systems

DBMS is a software system that manages, stores, and retrieves data efficiently in a structured format.

- It allows users to create, update, and query databases efficiently.
- Ensures data integrity, consistency, and security across multiple users and applications.
- Reduces data redundancy and inconsistency through centralized control.

History of DBMS

The first database management systems (DBMS) were created to handle complex data for businesses in the 1960s. These systems included Charles Bachman's Integrated Data Store (IDS) and IBM's Information Management System (IMS). Databases were first organized into tree-like structures using hierarchical and network models.

Component of DBMS

Hardware, Software, Data, Database Access Language, Procedures and Users all together form the components of a [DBMS](#).

Hardware

The hardware is the actual computer system used for keeping and accessing the database. The conventional DBMS hardware consists of secondary storage devices such as hard disks.

Software

Software is the actual DBMS between the physical database and the users of the system. All the requests from the user for accessing the database are handled by DBMS.

Data

It is an important component of the database management system. The main task of DBMS is to process the data. Databases are used to store the data, retrieved, and updated to and from the databases.

Users

There are a number of users who can access or retrieve the data on demand using the application and the interfaces provided by the DBMS.

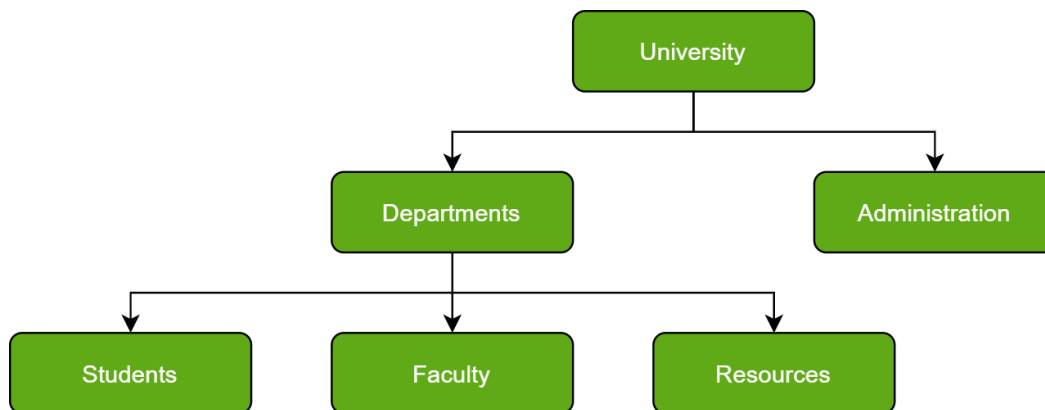
Database Access Language

Used to interact with the database (create, read, update, delete data). Examples: SQL, MyAccess, Oracle PL/SQL. DDL (Data Definition Language) – CREATE, ALTER, DROP DML (Data Manipulation Language) – INSERT, UPDATE, DELETE

Type of DBMS

1. Hierarchical Databases

Hierarchical databases organize data in a tree-like structure, where each parent record can have multiple child records. This model works well for scenarios where data follows a predefined hierarchical relationship, where data is arranged in levels or ranks.



Example: IBM's Information Management System (IMS) is a well-known hierarchical database.

2. Network Databases

A network databases build on the hierarchical model but allow child records to be linked to multiple parent records, creating a web-like structure of interconnected data. This results in a more flexible structure, often referred to as a graph model, where entities can be connected in many different ways.

Example: The Integrated Data Store (IDS) is a well-known example of a network database.

3. Object-Oriented Databases

Object-oriented databases are based on the principles of object-oriented programming (OOP), where data is stored as objects. These objects include attributes (data) and methods (functions), making them easily referenced and manipulated. These databases are designed to handle complex data structures such as multimedia, graphics, and large files.

4. Relational Databases Relational

databases are the most widely used type of database today. They store data in tables, with rows representing records and columns representing attributes of the records. In these databases, every piece of information has a relationship with every other piece of information. This is on account of every data value in the database having a unique identity in the form of a record. Note that all the data are tabulated in this model.

Example: MySQL, PostgreSQL, and Oracle Database are some popular relational databases.

5. Cloud Databases

A cloud database operates in a virtual environment hosted on cloud computing platforms. It is designed for storing, managing, and executing data over the internet, providing flexibility and scalability. Cloud databases are widely used for applications requiring dynamic workloads, as they eliminate the need for on-premises infrastructure.

6. Centralized Databases

A centralized database is a database stored and managed at a single location, such as a central server or data center. It ensures higher security and consistency as all data are maintained in one place, making it easier to control and manage.

7. Personal Databases

A personal database is a small-scale database designed for a single user, typically used on personal computers or mobile devices. These databases are ideal for managing individual data like contacts, budgets, notes, or schedules. They are lightweight, easy to use, and require minimal database administration, making them accessible for non-technical users.

8. Operational Databases

An operational database is designed to manage and process real-time data for daily operations within organizations and businesses. It allows users to create, update, and delete data efficiently, ensuring that the database reflects current activities and transactions.

9. NoSQL Databases

A NoSQL database (short for "non-SQL" or "non-relational") provides a mechanism for storing and retrieving data that does not rely on traditional table-based relational models. Instead, it uses flexible data models like key-value pairs, documents, column families, or graphs, making it ideal for handling unstructured, semi-structured, and structured data.

NoSQL databases are known for their simplicity of design, horizontal scalability (adding more servers for scaling), and high availability. Unlike relational databases, their data structures allow

faster operations in certain use cases. MongoDB, for instance, is a widely used document-based NoSQL database.

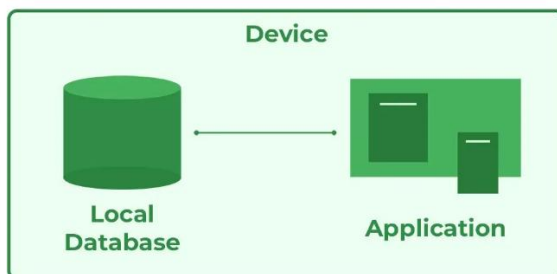
DBMS Architecture

The architecture of a Database Management System (DBMS) describes the structure and interaction of its components, as well as how data is stored, managed, and accessed. There are two main ways to explain DBMS architecture: based on system design (tiers) and based on levels of abstraction (ANSI/SPARC model).

Types of DBMS Architecture

1-Tier Architecture

In 1-Tier Architecture, the user works directly with the [database](#) on the same system. This means the client, server and database are all in one application. The user can open the application, interact with the data and perform tasks without needing a separate server or network connection.



Advantages of 1-Tier Architecture

Simple Architecture:

1-Tier Architecture is the most simple architecture to set up, as only a single machine is required to maintain it.

Cost-Effective:

No additional hardware is required for implementing 1-Tier Architecture, which makes it cost-effective.

Easy to Implement:

1-Tier Architecture can be easily deployed and hence it is mostly used in small projects.

Disadvantages of 1-Tier Architecture

Limited to Single User: Only one person can use the application at a time. It's not designed for multiple users or teamwork.

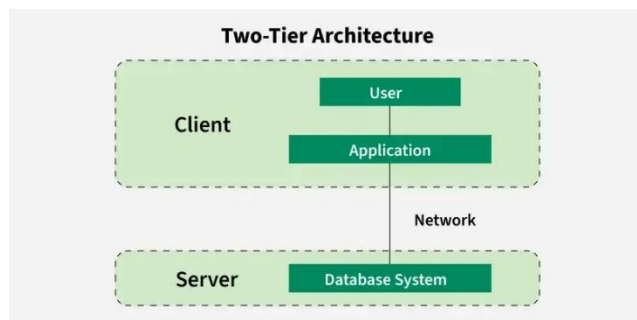
Poor Security: Since everything is on the same machine, if someone gets access to the system, they can access both the data and the application easily.

No Centralized Control: Data is stored locally, so there's no central database. This makes it hard to manage or back up data across multiple devices.

Hard to Share Data: Sharing data between users is difficult because everything is stored on one computer.

2-Tier Architecture

The 2-tier architecture is similar to a basic client-server model. The application at the client end directly communicates with the database on the server side. APIs like ODBC and JDBC are used for this interaction.



Advantages of 2-Tier Architecture

Easy to Access: 2-Tier Architecture makes easy access to the database, which makes fast retrieval.

Scalable: We can scale the database easily, by adding clients or upgrading hardware.

Low Cost: 2-Tier Architecture is cheaper than 3-Tier Architecture and Multi-Tier Architecture.

Easy Deployment: 2-Tier Architecture is easier to deploy than 3-Tier Architecture.

Simple: 2-Tier Architecture is easily understandable as well as simple because of only two components.

Disadvantages of 2-Tier Architecture

Limited Scalability: As the number of users increases, the system performance can slow down because the server gets overloaded with too many requests.

Security Issues: Clients connect directly to the database, which can make the system more vulnerable to attacks or data leaks.

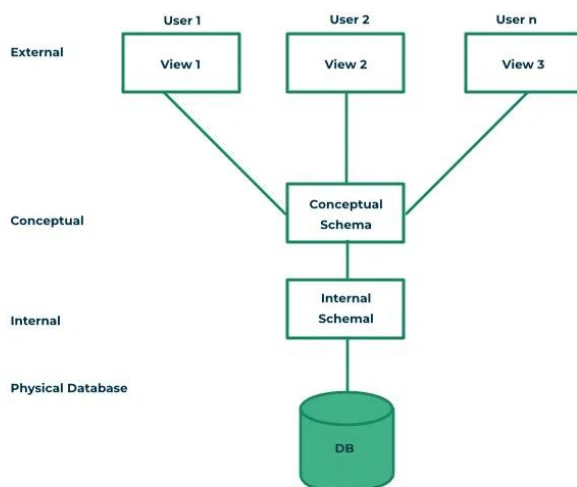
Tight Coupling: The client and the server are closely linked. If the database changes, the client application often needs to be updated too.

Difficult Maintenance: Managing updates, fixing bugs, or adding features becomes harder when the number of users or systems increases.

3 Tier Architecture in DBMS

In DBMS, the 3-tier architecture is a **client-server architecture** that separates the **user interface**, **application processing**, and **data management** into three distinct tiers or layers. The 3-tier architecture is widely used in modern web applications and enterprise systems because it offers scalability, flexibility, and security. Here is a brief description of each tier in the 3-tier architecture:

1. **Presentation Tier (User Interface Layer)**
2. **Application Tier (Business Logic Layer)**
3. **Data Management Tier (Database Layer)**



Components of the 3-Tier Architecture

1. Presentation Tier (User Interface Layer)

The presentation tier is the user interface or client layer of the application. It is responsible for presenting data to the user and receiving input from the user. This tier communicates with the Application Tier to process user requests and display relevant information. This tier can be a web browser, mobile app, or desktop application.

Example: If you're using a banking app, the presentation tier would display your account balance, allow you to make transfers, and display results based on your actions.

2. Application Tier (Business Logic Layer)

The application tier is the middle layer of the 3-tier architecture. It acts as the intermediary between the Presentation Tier and the Data Management Tier. It is responsible for processing and managing the business logic of the application. This tier communicates with the presentation tier to receive user input and communicates with the data management tier to retrieve or store data. This tier may include application servers, web servers, or APIs.

Example: If you're purchasing an item through an e-commerce platform, the Application Layer handles all the logic like verifying available stock, applying discounts, calculating taxes, and confirming your payment method.

3. Data Management Tier (Database Layer)

The Data Management tier is the bottom layer of the 3-tier architecture. It is responsible for managing and storing data. This tier can include databases, data warehouses, or any other persistent data storage solution. The data management tier communicates with the application tier save, retrieve, or manipulate data according to the business logic.

Example: In a customer relationship management (CRM) system, this tier would store customer details, transaction history, and other data relevant to the business.

Introduction to Data Modeling

A Data Model in Database Management System (DBMS) is the concept of tools that are developed to summarize the description of the database. Data Models provide us with a transparent picture of data which helps us in creating an actual database. It shows us from the design of the data to its proper implementation of data.

Types of Relational Models

1. Conceptual Data Model

The conceptual data model describes the database at a very high level and is useful to understand the needs or requirements of the database. It is this model, that is used in the requirement-gathering process i.e. before the Database Designers start making a particular database. One such popular model is the entity/relationship model (ER model). The E/R model specializes in entities, relationships, and even attributes that are used by database designers

Characteristics of a conceptual data model

This type of Data Models are designed and developed for a business audience.

The conceptual model is developed independently of hardware specifications like data storage capacity, location or software specifications like DBMS vendor and technology. The focus is to represent data as a user will see it in the “real world.”

Conceptual data models known as Domain models create a common vocabulary for all stakeholders by establishing basic concepts and scope

2. Representational Data Model

This type of data model is used to represent only the logical part of the database and does not represent the physical structure of the database. The representational data model allows us to focus primarily, on the design part of the database. A popular representational model is a Relational model. The relational Model consists of Relational Algebra and Relational Calculus. In the Relational Model, we basically use tables to represent our data and the relationships between them.

Characteristics of Representational Data Model

Represents the logical structure of the database.

Relational models like Relational Algebra and Relational Calculus are commonly used.

Uses tables to represent data and relationships.

Provides a foundation for building the physical data model.

3. Physical Data Model

The physical Data Model is used to practically implement Relational Data Model. Ultimately, all data in a database is stored physically on a secondary storage device such as discs and tapes. This is stored in the form of files, records, and certain other data structures. It has all the information on the format in which the files are present and the structure of the databases, the

presence of external data structures, and their relation to each other. Here, we basically save tables in memory so they can be accessed efficiently. In order to come up with a good physical model, we have to work on the relational model in a better way. Structured Query Language (SQL) is used to practically implement Relational Algebra.

This Data Model describes HOW the system will be implemented using a specific DBMS system. This model is typically created by DBA and developers. The purpose is actual implementation of the database.

Characteristics of a physical data model:

The physical data model describes data need for a single project or application though it maybe integrated with other physical data models based on project scope.

Data Model contains relationships between tables that which addresses cardinality and nullability of the relationships.

Developed for a specific version of a DBMS, location, data storage or technology to be used in the project.

Columns should have exact datatypes, lengths assigned and default values.

Primary and Foreign keys, views, indexes, access profiles, and authorizations, etc. are defined

ER Model

<https://www.geeksforgeeks.org/dbms/introduction-of-er-model/>

Relational Model

The Relational Model organizes data using tables (relations) consisting of rows and columns.

- The relational model represents how data is stored and managed in Relational Databases where data is organized into tables, each known as a relation.
- Each row of a table represents an entity or record and each column represents a particular attribute of that entity.
- The relational model transforms conceptual designs from ER diagrams into implementable structures. These structures are used in relational database systems like Oracle SQL and MySQL.

- The Relational Model is the most widely used model in database systems. It was proposed by E. F. Codd in 1970 while working at IBM. In this model, data is represented in the form of tables (called relations), which makes it simple, flexible, and easy to understand. The relational model is the foundation of most modern DBMSs such as Oracle, MySQL, PostgreSQL, Microsoft SQL Server, and others.

Key Terms in the Relational Model

1. **Attribute:** Attributes are the properties that define an entity. For example, ROLL_NO, NAME, ADDRESS etc.
2. **Relation Schema:** A relation schema defines the structure of the relation and represents the name of the relation with its attributes. For example, STUDENT (ROLL_NO, NAME, ADDRESS, PHONE and AGE) is the relation schema for STUDENT. If a schema has more than 1 relation it is called Relational Schema.
3. **Tuple:** A Tuple represents a row in a relation. Each tuple contains a set of attribute values that describe a particular entity. For example, (1, RAM, DELHI, 9455123451, 18) is a tuple in the STUDENT table.
4. **Relation Instance:** The set of tuples of a relation at a particular instance of time is called a relation instance. It can change whenever there is an insertion, deletion or update in the database.
5. **Degree:** The number of attributes in the relation is known as the degree of the relation. For example, The STUDENT relation has a degree of 5, as it has 5 attributes.
6. **Cardinality:** The number of tuples in a relation is known as cardinality. For example, The STUDENT relation defined above has cardinality 4.
7. **NULL Values:** The value which is not known or unavailable is called a NULL value. It is represented by NULL. For example, PHONE of STUDENT having ROLL_NO 4 is NULL.

Key in Relational Model

<https://www.geeksforgeeks.org/dbms/types-of-keys-in-relational-model-candidate-super-primary-alternate-and-foreign/>